

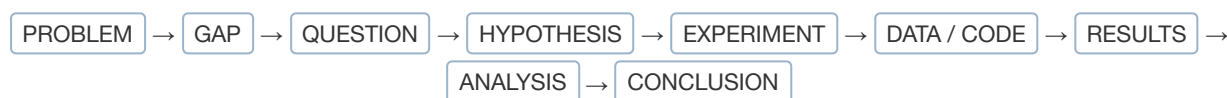
Think Less, Code Just as Well: Training a Small Reasoning Model to Think Shorter on Code, Compared with Simply Switching Its Thinking Off

Short title: Think Less, Code Just as Well

September 2026 · Planned duration: 12 weeks · Budget: \$0 (free Colab / Kaggle GPUs and free Hugging Face models and datasets)

THIS PROPOSAL IN SIMPLE WORDS

Some new AI models "think" (write hidden notes) before they answer. On code questions they often think far too long, even for easy questions. This costs time and computer power. We will teach one small, free AI model to think **shorter** on code questions **without** getting more answers wrong. Then we check a simple question: is this training really better than just pressing the model's built-in "thinking OFF" switch?



The research chain this proposal follows. Sections 2–3 cover the problem and gap, Section 4 the questions and hypotheses, Sections 5–7 the experiment.

1. Abstract

IN SIMPLE WORDS

Small "thinking" models waste words on code. We train one to use fewer words, and we test it fairly against the free, easy options (thinking OFF, a word limit, and a "think briefly" instruction).

A **large language model (LLM)** is a computer program that has learned to write text by guessing the next word, again and again. **Reasoning models** are LLMs that first write a hidden "thinking" part and then give the answer. Thinking often improves accuracy, but it is long: every extra **token** (a small piece of text, about $\frac{3}{4}$ of a word) costs time and computing power. Research calls this problem **overthinking**: long thinking where a short one would do.

Many 2026 small models have a built-in **thinking switch**: thinking can be turned ON or OFF. This gives a cheap way to save tokens without any training. Past work has trained models to think shorter, but mostly on math, mostly on older 7-billion-parameter models without a switch, and not against the switch itself.

This thesis uses one small, free 2026 model with a thinking switch (Gemma-4-E4B, with Qwen3.5-4B as a backup). The method is simple: the model answers each training code problem 4 times; we grade every answer with the problem's real tests; we keep the **shortest correct answer**; and we train a small add-on (**LoRA**) on these short correct answers. We then compare five ways of answering on

the same test problems: thinking OFF, a thinking budget, a "think briefly" prompt, normal thinking ON, and thinking ON with our trained add-on.

We expect (as hypotheses, not results) that the trained model will use at least 25% fewer thinking tokens while staying within 3 accuracy points of normal thinking, and will beat the no-training options. We also measure how many training examples are needed (100 to 2,000) and whether shorter thinking learned on code carries over to math. All work runs on free GPUs. Code, raw model outputs and trained add-ons will be released openly.

2. Motivation and Problem Statement

IN SIMPLE WORDS

Long thinking costs money and time. A free switch already exists. We found no study that checks whether training is worth the effort compared with that switch, on code, on a small new model.

EVERYDAY EXAMPLE

A student writes five pages for every exam question, even " $2 + 2$ ". We want to teach the student to write short answers when short is enough, without getting more questions wrong. We also check whether simply telling the student "don't think, just answer" works just as well. If it does, the teaching was not needed.

Code generation is one of the most used applications of LLMs. Reasoning models help on coding problems, but their thinking can be thousands of tokens long, even for easy problems. For a small model running on cheap hardware, this makes answers slow and expensive.

Our reading of the literature (Section 3, checked on 13 September 2026) shows three open gaps and two smaller ones:

Gap	What is missing	What this thesis does
G-A (main)	On one small model with a thinking switch, no study found compares thinking OFF, a thinking budget, a "think briefly" prompt, thinking ON, and thinking ON + shortest-correct training, on code.	The main experiment: <i>is training better than flipping the switch?</i>
G-B	No learning curve found showing how many examples shortest-correct training needs.	Train on 100 / 250 / 500 / 1,000 / 2,000 examples and draw the curve.
G-C	Shorter thinking learned on math has been shown to carry over to code. The opposite direction (code → math) was not found.	Train on code, test on math.
G-D (small)	One study found LoRA lost about 7 points compared with full training.	Compare LoRA rank 8 with rank 32.
G-E (small)	Little evidence on whether savings differ on problems the model may have seen before vs truly new problems, for a 2026 model.	Compare savings on problems from before and after the model's training cutoff date.

"Not found" means we did not find such a paper in our search. It does not prove that none exists. The search will be repeated before the related-work chapter is written.

3. Related Work and Gap Analysis

IN SIMPLE WORDS

Others have already made models think shorter. But they worked mainly on math, on older models, and did not compare against the thinking switch. That empty space is where this thesis sits.

The table lists the works closest to this thesis. The last column is the important one: what each work does *not* do.

Work	What it did	What it does not do (our gap)
Self-Training Elicits Concise Reasoning (Munkhbat et al., ACL Findings 2025)	Our exact method on math: sample many answers, keep the shortest correct one, fine-tune on it. Fewer tokens with accuracy about the same.	Math only. Full fine-tuning on large GPUs. The number of training examples was not varied, so there is no data-size curve (G-B).
SEER (arXiv 2509.14093, 2025)	Closest to us: shortest-correct answers plus a length filter, on software-engineering code data, with a 7B model. About 40% shorter reasoning. Found LoRA reached 67.7% vs 74.9% for full fine-tuning.	Older always-thinking 7B model with no switch, so no comparison with thinking OFF (G-A). No code → math test (G-C). The LoRA gap motivates G-D.
ASAP (arXiv 2508.05988, 2025)	Cuts unneeded steps out of code reasoning, then trains on them. 23.5% fewer tokens on LiveCodeBench.	Older 7–8B models without a switch; full fine-tuning; no switch baseline.
NoThinking (arXiv 2504.09858, 2025)	Showed that skipping thinking can beat thinking with a small budget, including on coding.	No training for shorter thinking. It shows why thinking OFF is a serious baseline we must beat.
HRBench (arXiv 2605.28398, 2026)	Compares ways of using the thinking switch on models from Qwen3.5-2B upwards; no single way wins.	Compares switch strategies, not a model trained on its own shortest correct answers against the switch (G-A).
One-Domain-to-All (arXiv 2601.06052, 2026); also HAPO, LC-R1	Shorter thinking trained on math carries over to code.	Only math → code. Code → math not found (G-C).

A warning we build in. S3-CoT (arXiv 2602.01982) reports that training only on the very shortest answers can hurt accuracy. For this reason our selection rule does not pick answers shorter than half of the typical correct length (Section 5.3). The paper "The Art of Efficient Reasoning" (arXiv 2602.20945) warns of a similar "short-is-correct trap".

What this thesis does NOT claim. To stay honest, we do not claim to be the first to shorten reasoning on code (SEER and ASAP did this), the first to do it on cheap GPUs (TokenSkip used LoRA on two consumer GPUs), or the first to show transfer between math and code (math → code is already shown). Our claim is narrower and checkable: a fair, same-model comparison against the thinking switch, a data-size curve, and the code → math direction.

4. Research Questions and Hypotheses

IN SIMPLE WORDS

Four questions: Is training better than the switch? How much data is needed? Does it work on math too? Does it still work on brand-new problems, and does the model still think longer when a problem is harder?

4.1 Research questions

1. **RQ1 (Training vs switch):** Does training a small hybrid-thinking model on its own shortest correct code answers give a better accuracy-for-tokens trade-off than thinking OFF, a thinking budget, or a "think briefly" prompt?
2. **RQ2 (How much data):** How do token savings and accuracy change as the number of training examples grows from 100 to 2,000?
3. **RQ3 (Transfer):** Does shorter thinking learned on code carry over to math problems?
4. **RQ4 (Fresh problems and adaptivity):** Do the savings also hold on problems published after the model's training cutoff, and does the trained model still think longer on medium problems than on easy ones?

4.2 Hypotheses

A **hypothesis** is a clear guess that the experiment can prove wrong. Each one below has a number, so there is no room to "wish" a result. "Thinking ON" means the normal, untrained model with thinking switched on. **pass@1** means the share of problems solved on a single try (we average over 4 tries).

ID	Hypothesis	Answers
H1	The trained model uses at least 25% fewer thinking tokens than thinking ON on the code test set.	RQ1
H2	Its accuracy (pass@1) stays within 3 points of thinking ON, measured on the same problems (paired).	RQ1
H3	It is more accurate than thinking OFF, the thinking budget and the "think briefly" prompt, at equal or fewer tokens.	RQ1
H4	(Exploratory) Token savings level off by about 1,000 training examples.	RQ2
H5	On math, thinking tokens also fall, with an accuracy loss of no more than 3 points.	RQ3
H6	The token savings point the same way on problems published after the model's cutoff.	RQ4
H7'	The trained model still thinks longer on medium than on easy problems, and its medium-problem accuracy stays within 3 points.	RQ4

What supports the main claim: H1, H2 and H3 all hold on the code test set.

What contradicts it: thinking OFF or the "think briefly" prompt does as well as the trained model,

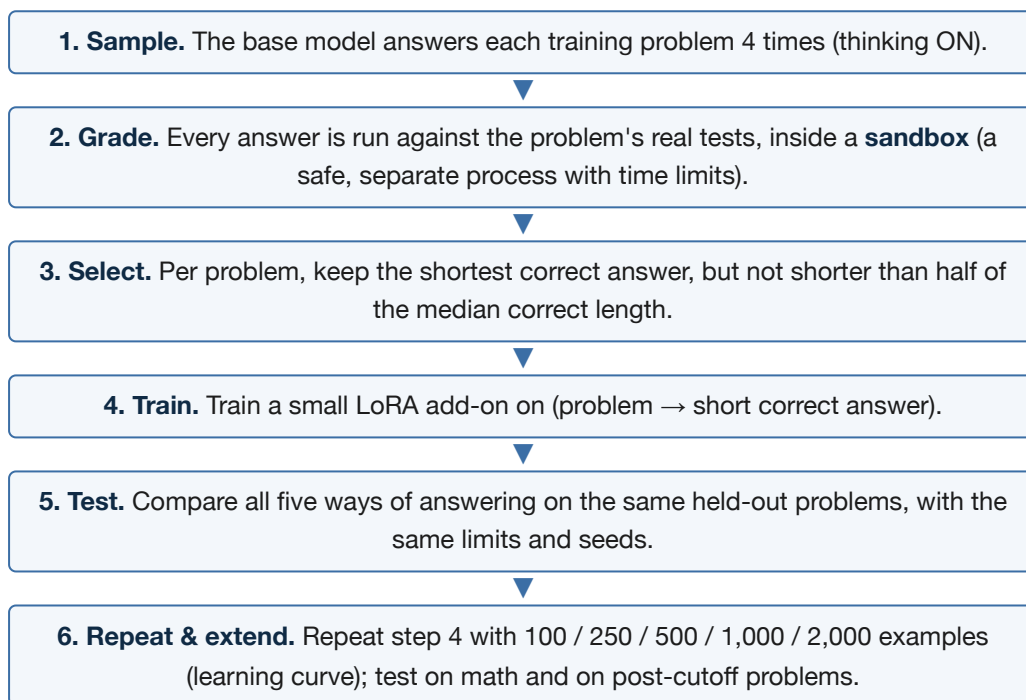
or accuracy drops by more than 3 points. This would still be a useful, honest result: it would tell practitioners that training is not worth the effort.

5. Proposed Method

IN SIMPLE WORDS

Let the model answer each practice problem 4 times. Check which answers really work. Keep the shortest one that works. Teach the model with those short, working answers. Then test it fairly against the free options.

5.1 The pipeline



Fine-tuning means giving an already-trained model a little extra training to teach it a new habit. **LoRA** (Low-Rank Adaptation) is a cheap way to fine-tune: instead of changing all of the model's numbers, we train a small add-on and leave the original model frozen. This fits on a free GPU and keeps the model's original abilities intact.

5.2 The five ways of answering (the "policies")

Policy	What it means	Why it is included
1. Thinking OFF	The built-in switch is turned off; the model answers directly.	The cheapest free option. Prior work shows it can be strong.
2. Thinking budget	Thinking is stopped after a fixed number of tokens, then the model must answer.	A free, simple way to force shorter thinking.
3. "Think briefly" prompt	Thinking ON, with an instruction to keep thinking short.	A free option that uses only words, no training.
4. Thinking ON	The normal, untrained model.	The main reference point for accuracy and tokens.

Policy	What it means	Why it is included
5. Thinking ON + our LoRA	The model with our trained add-on.	Our method.

5.3 Why the selection rule has a lower limit

If we only ever show the model the very shortest answers, it may learn "always stop early", even on problems that need more thinking (the S3-CoT warning in Section 3). Three protections are built in: (1) LoRA changes only a small add-on, so the model's long-thinking ability stays; (2) the selection rule skips answers shorter than half of the median correct length, so training shows a range of lengths; (3) hypothesis H7' checks directly whether the model still thinks longer on harder problems.

5.4 Stretch goal (only if time allows)

A second training method, **GRPO** (a kind of reinforcement learning, where the model is rewarded for good answers), with a length penalty applied **only to correct answers**. A 2026 study (Acoer, arXiv 2606.22716) reports that also penalising the length of wrong answers made training collapse, so we follow that warning. This is optional and not needed to answer RQ1–RQ4.

6. Experimental Design

IN SIMPLE WORDS

One small free model. Thousands of practice code problems. About 1,000–2,000 test problems it never trains on. Math problems for the transfer test. Clear rules so every policy is tested the same way.

6.1 Model

Role	Model	Why
Main model	google/gemma-4-E4B-it (March 2026)	Has a thinking ON/OFF switch; Apache-2.0 licence; its training data cutoff is published as January 2025, so problems from 2025 onwards are provably new to it; a free-GPU training notebook exists.
Backup (if the memory check fails)	Qwen/Qwen3.5-4B (February 2026)	Thinking switch, Apache-2.0, lower measured training memory. Downside: no published cutoff date.

How a 16 GB model fits on a 15 GB GPU. A model is a very long list of numbers. Gemma-4-E4B has about 8 billion of them. Stored at 16-bit (2 bytes each) the file is about 16 GB, too big for a free T4 GPU (15 GB). We load a **4-bit** version instead, where part of the numbers are stored in a squeezed form, like saving a photo as a JPEG: a little less detail, much smaller. That file is about 11 GB, and training on short examples has been measured at about 10.7 GB peak memory on a T4.

△ To be re-checked in weeks 1–2: the model's cutoff date on its model card, and whether the fast generation software (vLLM) runs this model on a T4. A known Gemma-4 number-overflow issue on T4 affects the audio part, so we use text only.

6.2 Data

Role	Dataset	Notes
Training (code)	DeepCoder-Preview-Dataset (primeintellect and taco parts, MIT) and APPS introductory problems (MIT)	Thousands of problems with tests. The lcbv5 part of DeepCoder is removed because it overlaps our test set. Training examples are kept at $\leq 3,500$ tokens.
Test (code, standard)	HumanEval+ (164), MBPP+ (378), LiveCodeBench easy + medium problems from before the cutoff	About 1,000+ problems in total (LiveCodeBench counts to be verified). The model may have seen some, but the base and trained models are compared on the same problems, so the comparison stays fair.
Test (code, fresh)	LiveCodeBench easy + medium problems dated after the model's cutoff	Provably unseen if the cutoff date is correct. Used for H6.
Transfer (math)	MATH-500, AIME 2026, HMMT February 2026	About 560 problems. Used for H5.

Overlap check. Before any training, every training problem that matches a test problem, exactly or nearly, is removed. A **benchmark** is a fixed set of test problems that many researchers use, so results can be compared. **Contamination** means test problems leaked into training data, which would make results look better than they are.

Why 1,000–2,000 test problems? A 2–3 point accuracy change is small. When the same problems are compared before and after training, roughly 1,250–1,900 problems are needed to see a 2.5-point change reliably. A large token change (-25%) is visible with a few hundred problems.

6.3 Scope: easy and medium problems

We train and test on easy and medium code problems. Hard problems are a stated limitation, for four reasons: hard-problem thinking is often 10,000–15,000 tokens, but training on a free T4 fits about 3,500; a 4-billion-parameter model solves few hard problems, so there are few correct answers to learn from; generating long answers for thousands of problems is too slow on free GPUs; and if the base model scores near 0% on hard problems, "did accuracy drop?" cannot be measured.

Pre-set exception: the first pilot also tries 50 hard problems. If the base model solves at least 15% of them, a small hard-problem check is added at the end. Otherwise hard problems stay a limitation.

6.4 Metrics

- **pass@1** (main accuracy measure): share of problems solved, averaged over 4 tries per problem.
- **Thinking tokens** (main cost measure): average length of the thinking part.
- **Limit-hit rate:** share of answers that run into the thinking limit.
- **Statistics:** paired **bootstrap** 95% confidence intervals, clustered by problem (bootstrap = re-sampling the results many times to see how much the numbers could vary by chance), with fixed random seeds.
- **Pareto chart:** accuracy vs thinking tokens for all policies on one chart. A policy is better if it is higher (more accurate) and further left (fewer tokens).

6.5 Fairness rules (the same for every policy)

- The same thinking limit at test time (for example 8,000 tokens) for every policy. The 3,500-token cap applies only to *training examples*.
- The same prompts (except the "think briefly" prompt being tested), the same seeds, the same problems.
- Answers are graded only with the benchmark's real tests, run in a sandbox, never by eye.
- All raw model outputs (thinking and answer) are stored, so any result can be re-graded later without re-running the model.

6.6 Gates: checks before spending GPU days

No one can promise in advance that training will help. But we can check *before* the expensive steps whether there is something to learn. These gates are fixed now, so the data decides, not our wishes.

Gate	Test	If it fails
1. Memory	Train on 50 examples up to 3,500 tokens long. Peak GPU memory must be ≤ 14 GB.	Switch to the backup model, Qwen3.5-4B. H6 then becomes "likely unseen" instead of "provably unseen".
2. Headroom	The base model answers 200 training problems 4 times each. Coverage (share of problems with at least one correct answer) must be $\geq 40\%$. Headroom (shortest correct $\div$ average correct length) must be ≤ 0.75 , meaning short answers are at least 25% shorter.	Change problem difficulty or the method <i>before</i> scaling up.
3. Hard problems	Same pilot, 50 hard problems. Base model must solve $\geq 15\%$.	Hard problems stay a written limitation.

7. Implementation Plan and Deliverables

IN SIMPLE WORDS

Twelve weeks: set up and check, run the gates, measure the free options, build the training data, train, test on new problems and math, then write.

7.1 Deliverables

1. **Notebooks** (Colab / Kaggle) for sampling, sandboxed grading, data selection, training and testing, each step explained.
2. **Dataset of raw outputs:** every model answer (thinking and final answer), its grade and its token count.
3. **Trained LoRA add-ons** for each data size (100 to 2,000) and both ranks (8 and 32).
4. **Results:** tables (CSV), the Pareto chart, the learning curve, and the transfer and fresh-problem results, each with a short written explanation.
5. **Thesis document** with method, results, discussion and limitations.

7.2 Timeline (12 weeks)

Weeks	Activity	Output
1–2	Set up free Colab/Kaggle environment. Re-check the model, its cutoff date and all dataset ids. Build the sandboxed grading step. First model call with thinking ON vs OFF. Finish the literature search.	Working pipeline on a few problems; verified model and data list
3	Run the gates: memory gate, headroom pilot (200 problems $\times$ 4 answers), 50 hard problems.	Gate report: go / switch model / adjust
4–5	Measure the baselines on the code test set: thinking OFF, thinking budget, "think briefly" prompt, thinking ON.	Baseline results table and first Pareto chart
6–7	Build the training data: sample 4 answers per training problem, grade, apply the selection rule. Run the overlap check.	Clean training set of short correct answers
8–9	Train LoRA add-ons on 100 / 250 / 500 / 1,000 / 2,000 examples; compare rank 8 vs 32; test each on the code test set.	Main comparison (H1–H3), learning curve (H4)
10	Test the best add-on on post-cutoff code problems and on math (MATH-500, AIME 2026, HMMT Feb 2026). Check adaptivity by difficulty.	Results for H5, H6, H7'
11–12	Statistical analysis, charts, writing, and final checks that all numbers match.	Final thesis and released materials

7.3 Resources and cost

Total cost: \$0. All computing uses free GPUs: Google Colab (sessions of up to about 12 hours) and Kaggle (about 30 GPU-hours per week on $2 \times T4$). Models and datasets come free from Hugging Face. No paid APIs, paid GPUs or subscriptions are used. A rough early estimate is many tens of GPU-hours in total (about 40 million tokens to build training data, plus testing about 7 policy settings on about $1,500 \text{ problems} \times 4 \text{ answers}$). This estimate will be replaced by measured numbers after the week-3 pilot. Work is saved to Google Drive about every 20 problems, because free sessions can end without warning.

8. Risks and Mitigations

IN SIMPLE WORDS

Things can go wrong. For each one, we already have a plan B, and a "bad" result still answers the research question.

Risk	Impact	Mitigation
Main model does not fit in free GPU memory	Medium	Memory gate in week 3; switch to Qwen3.5-4B (measured at 9.6 GB on a T4).
Not enough headroom (short correct answers are not much shorter)	High	Headroom gate before scaling up; change problem difficulty or method first.
Thinking OFF or the prompt does as well as training	Medium	This is a valid, honest answer to RQ1 ("training is not worth it"). The learning curve and transfer results still stand.

Risk	Impact	Mitigation
Model learns to "always stop early" and fails on medium problems	Medium	Selection rule with a lower limit; LoRA keeps the base ability; H7' checks this directly and any failure is reported.
LoRA is weaker than full fine-tuning (as seen in SEER)	Medium	Compare rank 8 vs 32; report the gap as a limitation of free-GPU training.
Free GPU sessions end or limits change	Medium	Save to Google Drive about every 20 problems; split work across Kaggle and Colab.
Software issues on the T4 (Gemma-4 16-bit overflow in the audio part; vLLM support)	Low–Medium	Text-only use; fall back to Unsloth or the <code>transformers</code> library (slower but works).
Test problems leaked into training data	Medium	Overlap check before training; separate test on post-cutoff problems.
Running model-written code is unsafe	Low	Code runs only inside a sandbox (separate process, time limits), never directly in the notebook.

9. Expected Contribution and Conclusion

IN SIMPLE WORDS

At the end we will know, with fair numbers: whether training is better than the free switch, how much data it needs, and whether it also works on math.

This thesis is expected to contribute:

1. **A fair comparison (G-A):** on one small 2026 hybrid-thinking model, shortest-correct LoRA training vs thinking OFF, a thinking budget and a "think briefly" prompt, on code, under identical rules.
2. **A learning curve (G-B):** how accuracy and token savings change from 100 to 2,000 training examples, which tells others how much data this method needs.
3. **Code → math transfer (G-C):** whether shorter thinking learned on code carries over to math.

The whole study runs on free hardware and all materials will be released, so anyone can repeat it. The result is useful in either direction: if training wins, it shows a cheap way to make small models faster on code; if the free switch wins, it saves others from training that is not needed. In both cases, the question is answered with evidence, and the limits (easy and medium problems, one small model, LoRA instead of full training) are stated clearly.

Glossary

LLM (large language model)	A program that learned to write text by guessing the next word from huge amounts of text.
Token	A small piece of text, about $\frac{3}{4}$ of a word. Models read and write tokens; more tokens means more time and cost.
Reasoning / thinking	Hidden notes a model writes before its final answer.
Overthinking	Writing long thinking where a short one would give the same answer.
Thinking switch (hybrid model)	A setting that turns the model's thinking ON or OFF.
Thinking budget	A hard limit on thinking tokens, after which the model must answer.
Fine-tuning	Extra training that teaches an existing model a new habit.
LoRA	A cheap fine-tuning method that trains a small add-on and leaves the main model unchanged. "Rank" is the size of the add-on.
GRPO	A reinforcement-learning method: the model tries answers and is rewarded for good ones.
Benchmark	A standard set of test problems used to compare models.
pass@1	The share of problems solved correctly on one try.
Sandbox	A safe, separate place with time limits where model-written code is run and tested.
Training cutoff	The date after which the model saw no training data. Problems published later are new to it.
Contamination	Test problems appearing in training data, which makes results look better than they are.
Transfer	A skill learned on one kind of problem (code) also working on another kind (math).
Learning curve	A chart of results against the amount of training data.
Pareto chart	A chart of accuracy against cost. The best options are those no other option beats on both.
Paired bootstrap / confidence interval	A way to measure how sure we are about a difference, by comparing the same problems and re-sampling the results many times.
GPU / T4	A GPU is a chip that runs AI models fast. The T4 is the free 15 GB GPU on Colab and Kaggle.
16-bit / 4-bit	How many bits store each of the model's numbers. 4-bit uses less memory with a small loss of detail.

References

1. Sui et al. "Stop Overthinking: A Survey on Efficient Reasoning for Large Language Models." TMLR, 2025. arXiv:2503.16419.
2. Munkhbat et al. "Self-Training Elicits Concise Reasoning in Large Language Models." ACL Findings, 2025. arXiv:2502.20122.

3. SEER: adaptive chain-of-thought compression for efficient reasoning in software-engineering tasks. 2025. arXiv:2509.14093.
4. "Pruning the Unsurprising: Efficient Code Reasoning via First-Token Surprisal" (ASAP). 2025. arXiv:2508.05988.
5. "Reasoning Models Can Be Effective Without Thinking" (NoThinking). 2025. arXiv:2504.09858.
6. HRBench: benchmarking hybrid reasoning (thinking-switch) strategies. 2026. arXiv:2605.28398.
7. S3-CoT: self-sampled variable-length chain-of-thought. 2026. arXiv:2602.01982.
8. "The Art of Efficient Reasoning." 2026. arXiv:2602.20945.
9. One-Domain-to-All: cross-domain transfer of efficient reasoning. 2026. arXiv:2601.06052.
10. HAPO: history-aware policy optimization for concise reasoning. 2025. arXiv:2505.11225.
11. LC-R1: length compression for reasoning models. 2025. arXiv:2506.14755.
12. TokenSkip: controllable chain-of-thought compression. EMNLP, 2025. arXiv:2502.12067.
13. Acoer: GRPO with LoRA for concise reasoning. 2026. arXiv:2606.22716.
14. "Demystifying Hybrid Thinking." 2025. arXiv:2510.12680.
15. Reasoning steps in thinking code LLMs. 2025. arXiv:2511.05874.
16. Gemma 4 Technical Report. 2026. arXiv:2607.02770.
17. Miller. "Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations." 2024. arXiv:2411.00640.
18. Jain et al. "LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code." livecodebench.github.io.

Note: some titles above are shortened descriptions taken from our search notes. Full titles, authors and every claim will be re-checked against the original papers before the related-work chapter is written.